What Software Systems Actually Receipt, Govern, and Hide

Findings from a 17-System Constitutional Corpus

Second, accountability surfaces are stratified rather than binary, ranging from mandatory operational ledgers to complete absence of receipts.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Modern software systems execute with authority, yet provide only partial accounts of how that authority is established, exercised, and constrained. This paper presents findings from a 17-system constitutional corpus spanning command-line tools, runtimes, databases, container platforms, orchestration systems, and a compiler. Using SFA, a ten-phase structural analysis method whose exact expansion remains an author decision, we examine how systems receipt actions, govern capabilities, and obscure internal decision processes.

Across the corpus, three stable findings emerge. First, authority is frequently derived from execution paths rather than fully declared in advance, though systems with explicit authority specification mechanisms provide counterexamples. Second, accountability surfaces are stratified rather than binary, ranging from mandatory operational ledgers to complete absence of receipts. Third, governance tends to concentrate at system boundaries while remaining fragmented in interiors, particularly in extension mechanisms and multi-layer policy compositions.

We replace binary claims with typed distinctions, demonstrate the method through contrasting examples, and show how adversarial analysis forced refinement of the claims. The paper concludes with concrete design implications and a defined next-stage falsification program.

Source, Method, and Evidence Standing

The exact paper source remains unchanged. This candidate binds its empirical claims to one supplied SFA corpus root containing the ten-phase methodology,

the 17-system post-adversarial synthesis, and seventeen per-system `ANALYSIS.md` records. The exact custody manifest is

`work/paper-saturation-v1/evidence/what-software-systems-actually-receipt-govern-and-hide/sfa-17-corpus-manifest.v0.1.json`,
SHA-256
`3108754C519D0BA248C868E77818C386C5F78BA0A6700EA76DEF8CFAFA8E59BD`.

The reviewed methodology names SFA **Semantic Defragmentation Analysis**. The source paper names it **Structural Fragmentation Analysis**, and another related paper uses **Semantic Fragmentation Analysis**. The current semantic index places SFA inside a wider Constitutional Software Analysis family. This candidate uses the acronym `SFA` without silently resolving those name and work-identity differences; the author must adjudicate them.

The evidence is internally authored and source-bounded. Exact hashes establish which artifacts were reviewed, not that their claims are correct, complete, independent, current, published, peer reviewed, or reproduced. This pass did not consult current vendor documentation or externally re-run the seventeen analyses. Product names and observations should therefore be read as corpus-time findings subject to version and environment drift.

1. Introduction

Software systems continuously make authorization decisions: what code runs, what data is accessed, what actions are permitted. These decisions are often difficult to explain, reproduce, or audit, even when systems expose configuration, logging, and policy mechanisms.

This paper asks:

What do software systems actually receipt, govern, and hide?

We answer empirically, using a corpus of 17 systems and a method - Structural Fragmentation Analysis (SFA) - designed to reveal how authority and accountability are distributed across real implementations.

The central result is not a universal theory, but a disciplined shift: binary claims fail. Typed distinctions are required.

1.1 Operative terms and claim geometry

Term	Meaning in this study
authority surface	a declared or implementation-derived locus that determines whether an actor or operation may produce an effect

Term	Meaning in this study
authority specification mechanism	a dedicated language or structure that declares authority separately from ordinary execution behavior
receipt surface	a record-producing surface that supports reconstruction of an operation, decision, provenance path, or effect within a stated scope
receipt coverage vector	the tuple of emission, operation scope, decision provenance, effect linkage, integrity, retention, correlation, and failure behavior; not a single yes/no property
governance boundary	a point at which a system explicitly mediates entry, authority, admissibility, or effects across a trust domain
interior governance	mediation of operations after entry rather than reliance on boundary admission alone
resolution cascade	an ordered or interacting set of precedence, lookup, default, override, or policy-selection rules
composition opacity	inability to reconstruct why the combined cascades and governance layers produced one observed outcome
extension surface	a hook, plugin, script, macro, module, recipe, or other mechanism that introduces externally authored behavior
bypass	a declared or implementation-derived mechanism that relaxes a named authority constraint; bypasses differ in scope and remaining controls
projection divergence	a scoped difference between a system's public or documented model and the operational structure found by the analysis

A finding should be represented as:

$$F = \langle s, v, q, k, e^+, e^-, b, \sigma \rangle,$$

where s is the system, v the analyzed version or snapshot, q the question and

scope, k the finding kind, e^+ supporting evidence, e^- counterevidence, b the boundary conditions, and σ the standing such as candidate, supported-within-corpus, weakened, replaced, or retired. This prevents a cross-system summary from outliving the scope and evidence that gave it meaning.

---

2. Methodology

2.1 SFA Procedure

Each system was analyzed using five steps:

1. Identify authority surfaces
2. Identify receipt surfaces
3. Trace resolution chains
4. Evaluate extension mechanisms
5. Compare projection vs execution

The source corpus applies these operations through documentation, source or architectural inspection, configuration, maps, and selected evidence excerpts. This candidate did not independently repeat every inspection against a live or current product version.

2.1.1 Corpus enumeration and evidence boundary

The seventeen analyses are: ripgrep, fzf, bat, uv, GNU Make, Cargo, npm, pytest, FastAPI, Neovim, Docker, Kubernetes, PostgreSQL, Vault, rustc, Git, and Deno. They span CLI tools, build and package systems, frameworks, an editor, infrastructure platforms, multi-user authority systems, a compiler, version control, and a capability-oriented runtime. They are not a random or representative sample of all software.

The exact SFA methodology is SHA-256 `E3F9526FDBD53ADA39F6A77310FF1EE9B55B42C2C7B90C76AD7F56616F123770`. The exact post-adversarial synthesis is SHA-256 `BAC6B926B910B644D460128E8B4EC97DCF3120E527FA544AD1D0C1FF616AEBBA`. The custody manifest lists each analysis path, byte length, and SHA-256.

The per-system records combine documentation, configuration, source or architectural inspection, maps, and selected evidence excerpts. Their methods and depth vary by system. The paper is therefore a source-bounded comparative study, not a controlled experiment, systematic review, benchmark, or security audit. Claims are strongest when they quote an exact analysis and name its scope; they weaken as they move toward universal software statements.

2.2 Worked Example: Vault (Strong Governance Case)

Vault exposes both authority and accountability explicitly.

Step 1 - Authority surfaces Authority is defined through policies attached to tokens. Each request is evaluated against policy rules specifying allowed capabilities on paths.

Step 2 - Receipt surfaces Vault produces mandatory audit logs. Each request generates a structured record including:

- request path and operation
- token identity
- policy evaluation (`policy_results.granting_policies`)
- response outcome

This is the strongest authorization-level receipt surface in the reviewed corpus, but it is not complete without qualification: root-token bypass, unseal/bootstrap, HMAC verification indirection, external audit-device trust, retention, and effects beyond the reviewed API scope remain separate.

Step 3 - Resolution chain The decision path is externally visible: request -> token -> policies -> capability match -> decision

Step 4 - Extension mechanisms Vault governs plugin registration and process placement, but the corpus does not find per-plugin capability declarations or complete extension-effect receipts. It is the closest observed case, not a fully governed extension model.

Step 5 - Projection vs execution Vault's model ("policies govern access") aligns with actual execution.

Result Vault demonstrates:

- declared authority
- mandatory API request/response audit within the analyzed scope
- authorizing-policy provenance for reviewed operations
- relatively low projection divergence
- explicit but still material bootstrap, bypass, plugin, and audit-device boundaries

---

2.3 Contrasting Example: npm (Weak Governance Case)

Applying the same steps to npm produces a fundamentally different structure.

Authority surfaces Authority emerges from execution: lifecycle scripts (`preinstall`,

`postinstall`) run arbitrary code with user privileges.

Receipt surfaces No unified mandatory authorization-and-effect receipt records which lifecycle scripts executed and what system effects they produced; resolution artifacts and console output are only partial surfaces.

Resolution chain Execution is implicit: dependency resolution -> script execution -> system effects, with no unified trace.

Extension mechanisms The analyzed lifecycle-script surface executes with ambient build-user authority and lacks a capability declaration or complete effect receipt.

Projection vs execution The interface suggests "install dependencies," but execution includes arbitrary code.

Result npm demonstrates:

- implementation-derived authority
- absent receipts
- ungoverned extensions
- high projection divergence

This contrast shows the dynamic range of SFA: the same procedure reveals radically different governance structures.

2.4 Adversarial Refinement

The analysis evolved through adversarial testing.

Initial claims (12 systems):

- configuration is authority
- receipts are incomplete
- authority chains follow resolution topology

Batch-3 introduced systems designed to falsify these claims.

Failures:

- "Configuration is authority" -> replaced with three typed relationships
- "Receipts are incomplete" -> replaced with four receipt tiers
- "Authority chains follow resolution topology" -> invalidated

Survivors:

- Implementation-derived authority (with boundary conditions)
- Projection divergence
- Resolution cascade opacity (strengthened)

The method is validated not by initial correctness, but by its capacity to refine under falsification pressure.

---

3. Authority: Declared vs Enacted

In 12 of the 17 analyzed systems, authority is substantially implementation-derived:

- npm lifecycle scripts
- Docker runtime configuration
- CLI tools combining flags and environment

Explicit authority appears only with dedicated mechanisms:

- Deno permissions (`--allow-read`, `--allow-net`) with audit mode (`DENO_AUDIT_PERMISSIONS`)
- PostgreSQL privilege system
- Vault policies
- rustc type system and `unsafe` boundaries

Boundary condition Authority is explicit only when a system provides a dedicated specification surface.

---

4. Accountability: Receipt Surface Stratification

Receipt surfaces fall into four tiers:

- **Tier:** Mandatory ledger
  Description: Reviewed API operations produce request/response audit records with policy provenance; audit failure blocks service
  Evidence: Vault audit logs (`policy_results.granting_policies`)
- **Tier:** Opt-in ledger
  Description: Full logging available but disabled
  Evidence: Kubernetes audit (`authorization.k8s.io/reason`)
- **Tier:** Gate-only
  Description: Failures explained, successes silent
  Evidence: rustc diagnostics
- **Tier:** Absent
  Description: Minimal or no receipts
  Evidence: CLI tools, npm

Observation Within this selected corpus, receipt completeness correlates with external accountability pressure more than with system complexity. This is a corpus hypothesis, not an established causal law.

---

5. Governance: Boundary vs Interior

Governance concentrates at boundaries:

- Docker: a default seccomp profile blocks a version-dependent subset of system calls
- Kubernetes: API server enforces RBAC and admission
- Git: remote operations governed

Interior execution is weaker:

- containers run broadly privileged processes
- Kubernetes workloads require additional policies for restriction
- build systems execute arbitrary code

Implication In the analyzed cases, boundary enforcement can coexist with interior zones whose authority and effects require separate evidence. The generality of this pattern remains under test, and the strongest interior-governance contrast is based mainly on Vault and PostgreSQL.

---

6. Resolution Cascade Opacity

The Kubernetes analysis provides an illustrative configuration-derived failure path. It is not presented here as an independently replayed exploit.

Scenario:

A Pod is created with `hostNetwork: true`.

- RBAC allows the user to create Pods
- Mutating webhooks do not modify the request
- Pod Security Standards are not enforced in the namespace
- No NetworkPolicy restricts the Pod

The Pod runs with host network access.

Question: which layer should have prevented this?

The audit log records only: `authorization.k8s.io/reason = "allowed by RBAC"`

It does not explain:

- absence of admission enforcement
- lack of network policy
- interaction between layers

Finding When governance is distributed across independent mechanisms, no unified explanation exists for outcomes.

Implication Systems require composable introspection surfaces capable of reconstructing full decision paths across layers.

---

7. Configuration and Authority

Three relationships replace the binary claim:

- Accidental entanglement: CLI flags encode authority implicitly
- Structural separation: PostgreSQL separates config and authorization
- Inherent entanglement: Kubernetes Pod specs define capabilities

This enables comparison without collapsing distinct system structures.

---

8. Extension Surfaces

Extension mechanisms:

- Ungoverned: npm scripts, Rust `build.rs`
- Perimeter-governed: Deno modules
- Fully governed: none observed

Critical observation Across the four analyzed build-time ecosystems–npm lifecycle scripts, Rust build scripts and procedural macros, Make recipes, and Git hooks–no fully governed extension surface was observed.

Implication Language-level guarantees alone do not govern these ambient build-time processes. A separate enforcement boundary–for example OS isolation, a capability runtime, or another independently enforced sandbox–is required for the claimed restriction, and its own bypass and receipt behavior must be tested.

---

9. Authority Bypass Spectrum

All six analyzed systems with non-trivial authority models include a bypass mechanism:

Type	Example
Unbounded	PostgreSQL SUPERUSER, Vault root token
Process	Docker `--privileged`, Deno `--allow-all`
Scoped	Rust `unsafe`

Within this six-system subcorpus, the distinction lies in scope and remaining controls, not mere presence. Six examples do not establish a universal invariant.

---

10. Projection Divergence

All seventeen analyses record some projection divergence, but only about ten classify it as structurally significant for governance reasoning; near-minimal cases include PostgreSQL, Vault, and rustc.

Examples:

- npm hides lifecycle execution
- Kubernetes exposes partial reasoning
- CLI tools obscure resolution chains

The divergence is universal but varies in impact.

---

11. Structural Causes of Hiding

Systems obscure:

- execution paths
- authority derivation
- layered decision composition
- extension behavior

These arise from structural pressures:

- cost of recording vs value of accountability
- performance constraints
- compositional complexity
- developer ergonomics

Proto-theoretical direction Accountability is consequence-shaped: systems expose what is costly to ignore and hide what is costly to explain.

---

12. Public Findings with Corpus Scope

1. Implementation-derived authority appears in 12/17 systems; all five counterexamples use a dedicated authority specification mechanism.
2. Receipt surfaces form four observed tiers, from one mandatory-ledger exemplar to absent or partial surfaces.
3. Clear resolution-cascade opacity appears in 7/17 systems; no analyzed multi-layer case provides one unified explanation surface.
4. Governance tends to concentrate at trust-domain boundaries in the observed corpus; interior-governance generalization remains based on limited exemplars.
5. Configuration-authority relations separate into accidental entanglement, inherent entanglement, and structural separation.

6. No fully governed extension surface was observed; four analyzed build-time ecosystems retain ambient execution authority.
7. Projection divergence appears in all 17 analyses but is structurally significant in about 10.
8. Bypass mechanisms appear in all six analyzed authority-bearing systems and range from unbounded to scoped and partially governed.

---

13. Universal GSM, GMB, and Corpus Integration

13.1 Governed semantic records for comparative software analysis

The universal governed semantic model should keep these entities distinct: analyzed system, version or snapshot, entry/exit surface, authority source, admissibility rule, capability, transformation, receipt, boundary, extension, bypass, projected claim, observed structure, evidence item, counterexample, finding, claim movement, and revision. A lexical mention of the same product or mechanism is not an identity relation between analyses.

The **Governed Meaning Base (GMB)** is the current name in the author-confirmed lineage `CCF -> CMB -> GMB`. GMB can retain each finding with exact evidence, limits, counterexamples, and standing movement. For example, a binary claim should not be overwritten when falsified; it should remain linked to its typed successor and the counterexample that forced revision. GMB admission does not make the finding true, current, or independently verified.

13.2 Direct corpus interfaces

- *Reasoning by Way of Receipts* clarifies that a receipt binds a claim to reasoning, evidence, and scope; it does not turn a log entry into a complete account of effects.
- *Semantic Debt* names the future burden created when decision paths, precedence, extension authority, or cross-layer composition remain implicit.
- *Semantic Legitimacy Before Context* requires evidence and version context to be admissible before it can support a system claim.
- *A Formal Theory of Work Custody* keeps product, analyzed artifact, analysis record, synthesis, paper, and publication identities separate.
- *Admissibility Engineering* turns proposed remedies into selectable profiles rather than assuming every system should adopt one universal governance shape.
- *The Integrity Enforcement Layer* provides a candidate enforcement vocabulary for validation, corruption, read-only inspection, refusal, and atomic or classified effects; it is not inferred to exist in the seventeen systems.

These interfaces deepen the analysis model. They are not additional observations of the seventeen products.

14. Design Implications

Unified receipt surfaces Layered systems should expose composable queries that reconstruct full decision paths.

Explicit authority declaration Separate authority from execution wherever possible.

Governed extension models Build-time execution must be sandboxed or explicitly governed.

Scoped bypass mechanisms Replace global overrides with constrained capabilities.

Interior governance alignment Extend boundary controls into execution contexts.

---

15. Realization Funnel for an SFA Study

Stage	Role
Possibility	Generate candidate system classes, versions, deployment modes, authority models, receipt types, extensions, bypasses, counterexamples, and harmful or null cases.
Specification	Freeze the SFA phases, taxonomy, corpus question, sampling rule, system/version identities, evidence requirements, scoring rules, claim statuses, and falsifiers.
CM1	Enumerate admissible study designs and corpora, including biased, uneven-depth, costly, obsolete, weakly reproducible, or low-information configurations.
CM2	Prefer diverse falsification targets, reproducible versions, comparable evidence depth, independent checks, explicit uncertainty, and minimal unsupported generalization.
Instance Configuration	Select the exact seventeen systems, snapshots, source roots, method version, analyst instructions, evidence files, analysis depth, and synthesis procedure.

Stage	Role
Implementation Realization	Perform the ten analysis phases, produce maps and evidence indices, adjudicate findings, run adversarial cases, and compile the synthesis.
Evidence	Preserve system records, source excerpts, maps, counterexamples, retired claims, case-level uncertainties, cross-system counts, and review outcomes.
Revision	Add or replace system cases, correct evidence, split categories, narrow claims, and retain the lineage from failed binary claims to typed distinctions.

The current paper has evidence from one Instance Configuration. It does not establish that its configuration is CM2-optimal or that its results transport to other software populations.

Constitutional Software Evidence Study

Across the completed P1 corpus, What Software Systems Actually Receipt, Govern, and Hide is the **constitutional software evidence study over one frozen seventeen-system configuration**. It operationalizes SFA categories against exact internal analyses, records where binary hypotheses broke, and returns bounded comparative findings. It is neither the SFA method itself nor a population estimate about software in general.

- **Adjacent work:** *Semantic Field Analysis*
  Supplies: A general procedure for mapping entities, authority, transformations, receipts, boundaries, projections, and hidden structure.
  Study boundary: This paper is one empirical application and evidence corpus.
- **Adjacent work:** *Semantic Fragmentation and Constitutional Structure*
  Supplies: A structural theory of why authority, meaning, state, and evidence separate across software layers.
  Study boundary: The seventeen-system study supplies scoped observations rather than universal proof.
- **Adjacent work:** *Reasoning by Way of Receipts*
  Supplies: Claim-bounded evidence records and revision lineage.
  Study boundary: The study distinguishes logs, traces, proofs, explanations, and effect evidence rather than counting all as receipts.

- **Adjacent work:** *Governance Coverage Gaps*
  Supplies: Detection of consequential capabilities and effects outside declared governance.
  Study boundary: SFA locates concrete gap patterns inside selected software structures.
- **Adjacent work:** *The Constitutional Observability Platform*
  Supplies: Role-aware projection of operational and evidentiary state.
  Study boundary: A projection can expose an analysis without repairing the system it describes.

Each case should be recoverable as:

$$C = (system, version, deployment, question, method, evidence, \\ authority_map, receipt_map, governance_map, \\ extension_map, bypass_map, projection_map, findings, \\ uncertainty, review)$$

Cross-system synthesis should derive only from case fields that meet a declared comparability rule. A missing surface can mean absence, insufficient instrumentation, inaccessible evidence, or analysis failure; these outcomes must not be counted alike. Unequal analysis depth, version drift, product-family variance, and internally correlated reviewers remain threats to every aggregate.

The study's most valuable result is methodological revision under counterexample: implementation-derived authority, receipt stratification, configuration entanglement, bypass scope, and projection divergence replaced earlier binary language. Later studies should preserve those claim movements and test them against deliberately selected falsification cases rather than treating 17 as a representative denominator.

The universal GSM can type `APPLIES_SFA_METHOD`, `ANALYZES_SOFTWARE_SNAPSHOT`, `OBSERVES_AUTHORITY_SOURCE`, `CLASSIFIES_RECEIPT_SURFACE`, `LOCATES_GOVERNANCE_BOUNDARY`, `FINDS_BYPASS_PATH`, `MEASURES_PROJECTION_DIVERGENCE`, `RETIRES_BINARY_FINDING`, `LIMITS_AGGREGATE_TO_CORPUS`, and `REQUIRES_PRIMARY_REVALIDATION`. These relations make the evidence study reusable without inflating its scope.

This integration establishes four editorial commitments: classify the paper as the seventeen-system constitutional software evidence study; adopt the recoverable case envelope and explicit missing-evidence outcomes; bind every aggregate to comparability and exact corpus scope; and preserve SFA method, fragmentation theory, this study, and any later 79-system study as separate linked works.

Versioned Evidence Cases Without Structural Overclaim

The fixed-seed comparison with CCOO, the Epistemic Lineage Engine, the MCIR DAG theorem, and Semantic Field Analysis separates the study's real interfaces from accidental adjacency:

$$\begin{aligned} C^* = (&case_identity, system_head, method_version,\\ &evidence_cut, analysis_operations, findings,\\ &alternatives, review, successor, aggregate_eligibility,\\ &remainder) \end{aligned}$$

ELE should preserve changes to each case, its source snapshot, analysis method, category decisions, counterexamples, and aggregate inclusion. This permits later reanalysis without rewriting the original finding or treating a new system version as the same observation.

An MCIR DAG can make internal case references closed and acyclic under a profile. That structural property does not establish source adequacy, correct classification, reviewer independence, or generalizability. SFA remains the diagnostic method; this paper remains one bounded evidence application whose claims depend on exact cases and comparability rules.

CCOO is an experimental collective architecture and has no direct evidentiary relation to the seventeen-system software corpus. The random neighborhood does not license importing its topology or toy results into the study.

The universal GSM can type
`LINEAGES_SOFTWARE_CASE`, `REANALYZES_SYSTEM_HEAD`,
`REFERENTIALLY_CLOSES_CASE_PACKAGE`, `APPLIES_SFA_UNDER_VERSION`,
`ELIGIBLE_FOR_BOUNDED_AGGREGATE`, `RETAINS_CASE_REMAINDER`,
and
`HAS_NO_MATERIAL_RELATION_TO_CCOO`.

This randomized review establishes four editorial commitments: adopt the versioned case object; preserve case and method lineage; keep DAG closure below evidentiary standing; and record the CCOO nonrelation.

A Reproducible Comparative Evidence Case

Five downstream uses rely on this study for ACG scope, quantitative evaluation, semantic debt, fragmentation analysis, and sovereignty barriers. The work should therefore publish its findings as versioned comparative evidence cases rather than timeless rankings of named systems.

$$E^* = (corpus_generation, system_identity, version,$$
$$environment, question, method, observer,$$
$$evidence_surface, declared_path, operative_path, \quad .$$
$$receipts, hidden_region, findings, counterexamples,$$
$$uncertainty, expiry, reproduction)$$

Absence from an observed receipt or governance surface is not evidence of global absence unless coverage and denominator are known. Conversely, a declared policy or receipt cannot establish that a consequential path is mediated. Product names, version families, plugins, hosted services, and local builds require separate identity bindings. Comparative labels must remain claim-specific: one system can expose authority well while hiding resolution, or govern mutation while poorly evidencing effects.

Reproduction needs the exact corpus snapshot, collection method, query set, classification rules, evidence excerpts, reviewer decisions, counterexamples, and known inaccessible surfaces. Later versions form successors; they do not silently overwrite earlier findings.

The universal GSM can type
`OBSERVES_SOFTWARE_EVIDENCE_CASE`, `COMPARES_CLAIM_SURFACE`,
`HIDES_RESOLUTION_REGION`, `LACKS_KNOWN_DENOMINATOR`, `EXPIRES_SYSTEM_FINDING`,
`REPRODUCES_COMPARATIVE_CASE`, and
`DOES_NOT_RANK_SYSTEM_GLOBALLY`.

This review establishes four editorial commitments: adopt the versioned evidence-case record; bind findings to exact versions and environments; prohibit global absence and ranking inferences from partial surfaces; and require reproducible excerpts, rules, counterexamples, and inaccessible-region disclosure.

16. Limitations

The corpus is limited to 17 systems and excludes:

- formally verified kernels
- large-scale IAM systems
- financial transaction platforms

Findings are constrained to observed domains, selected open-source systems, corpus-time versions and documentation, uneven per-system evidence depth, and one internally authored analysis program. The corpus omits proprietary, medical, embedded, mobile, financial, cloud-IAM, formally verified, and effect-tracked systems. The authority-bearing subset contains six systems; the strongest interior-governance comparison contains two; rustc is the only compiler. Counts describe this Instance Configuration and are not population estimates.

Publish Versioned Evidence Cases, Not Timeless Product Rankings

Public orientation

This paper compares seventeen internally analyzed software systems to ask where authority is declared, where actions are receipted, and where resolution remains hidden. The corpus is real and exactly custodied, but it is selected, uneven, internally authored, and tied to corpus-time versions and environments. Its strongest contribution is a reproducible set of scoped evidence cases, not a universal ranking of current products.

Adversarial tests

1. **Version-drift attack.** Product behavior, defaults, plugins, hosted services, documentation, and authority mechanisms changed after the analyzed snapshot.
2. **Absence-denominator attack.** No receipt or policy was observed and the paper infers global absence without knowing all relevant paths.
3. **Declaration-effect attack.** A documented control is counted as governance without testing whether consequential paths actually traverse it.
4. **Selection-bias attack.** Seventeen mostly open-source developer and infrastructure systems are treated as representative of software generally.
5. **Depth-variance attack.** Uneven per-system methods and evidence are compared as though every finding had equal coverage and confidence.
6. **Method-identity attack.** Semantic Defragmentation, Semantic Fragmentation, and Structural Fragmentation Analysis are silently merged under SFA.
7. **Reputation-harm attack.** Scoped historical findings are read as current security judgments, vendor evaluations, or claims of intentional opacity.
8. **Count-inflation attack.** Instance counts such as 12/17 or 6/6 are presented as population estimates, causal results, or independent replication.

Claim and evidence ceiling

The paper provides internally authored comparative evidence cases over one exact seventeen-system corpus. It is not a current vendor audit, security assessment, representative sample, independent replication, benchmark, controlled experiment, or global ranking. Product-specific findings expire outside their bound identity, version, environment, question, method, and observation surface.

Further evidentiary strengthening requires resolution of the SFA name and work relation, exact version and environment freezing, claim-by-claim source validation, current primary vendor documentation, reproduction rules and excerpts, counterexamples, inaccessible-region disclosure, and careful non-disparaging language. No external research was added in this pass.

Reader-facing consequence

The valuable public result is a way to compare exactly what selected software versions disclosed about authority and effects, while showing the evidence and unknown regions for every case instead of issuing a single trust score.

17. Publication Standing and Evidence Limits

This paper has a real internal evidence corpus, but it remains an internally authored comparative analysis of seventeen selected open-source systems. It is not an independent replication, current-vendor audit, representative sample, peer-reviewed software study, or universal theory. Product behavior, defaults, and documentation may have changed since the recorded analyses.

18. Conclusion

Software systems do not uniformly declare, receipt, or govern their behavior. They do so selectively, under structural pressure, and with consistent patterns of omission.

Typed distinctions make these patterns visible.

They allow precise statements: where authority is declared or derived, where accountability exists or collapses, where governance concentrates or fragments.

This replaces vague critique with actionable structure.

The next step is explicit: test these findings against systems designed to falsify them. Future analysis will target:

- Linux kernel MAC frameworks
- Cloud IAM systems
- mobile permission platforms
- formally verified systems

If the distinctions hold, they strengthen. If they break, they refine.

That is the discipline that produced these results - and the one required to extend them.

Realization Scope and Evidence Boundary

This successor makes the paper's stage boundary explicit. It narrows claims rather than manufacturing missing evidence: a stage named as context is not thereby claimed as implemented, validated, or complete.

- **Possibility Space.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.

Corpus-Wide Reference Positioning

This paper argues that modern software systems execute with authority, yet provide only partial accounts of how that authority is established, exercised, and constrained. Its nearest external comparison fields are provenance, software, open sovereign. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: A complete provenance graph can faithfully record a false or unauthorized process; provenance is evidence about lineage, not automatic truth or legitimacy. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. Source availability and reproducible builds improve inspectability but do not alone guarantee comprehensibility, practical exit, hardware control, or sustainable maintenance. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] W3C. *PROV-O: The PROV Ontology.* W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-o/ **Relationship:** Adjacent provenance

standard; it models entities, activities, and agents, but does not by itself establish the paper's stronger authority or standing claims.

[**E2**] Santiago Torres-Arias et al. "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes." *28th USENIX Security Symposium*, 2019, pp. 1393-1410. usenix.org source **Relationship:** Predecessor supply-chain lineage system used for comparison; no influence claim is made unless the source head already documents one.

[**E3**] OpenSSF. *Supply-chain Levels for Software Artifacts (SLSA), Version 1.2.* 2026. https://slsa.dev/spec/v1.2/ **Relationship:** Adjacent software-supply-chain assurance standard, especially relevant to provenance and verification boundaries.

[**E4**] OpenTelemetry Authors. *OpenTelemetry Specification.* https://opentelemetry.io/docs/specs/otel/ **Relationship:** Adjacent observability standard for traces, metrics, logs, and context propagation; telemetry is not equated with adjudicated evidence.

[**E5**] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15(12), 1972, pp. 1053-1058. DOI: 10.1145/361598.361623. https://doi.org/10.1145/361598.361623 **Relationship:** Foundational modularity treatment used to compare information hiding with explicit semantic and governance interfaces.

[**E6**] Ward Cunningham. "The WyCash Portfolio Management System." *OOPSLA '92 Experience Report*, 1992. DOI: 10.1145/157709.157715. https://doi.org/10.1145/157709.157715 **Relationship:** Original technical-debt metaphor used to distinguish implementation debt from semantic debt.

[**E7**] Karen Scarfone, Murugiah Souppaya, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1.* NIST SP 800-218, 2022. DOI: 10.6028/NIST.SP.800-218. https://doi.org/10.6028/NIST.SP.800-218 **Relationship:** Adjacent secure-development framework for preparation, protection, production, and vulnerability response.

Source-head external references retained under original labels [**E1**] *SFA Methodology: Semantic Defragmentation Analysis - Analytical Procedure*, Version 1.0, Directive SFA-STRUCTURE-1. Exact reviewed SHA-256 `E3F9526FDBD53ADA39F6A77310FF1EE9B55B42C2C7B90C76AD7F56616F123770`.

[**E2**] *SFA-SYNTHESIS-4: Post-Adversarial Consolidation*, March 17, 2026. Exact reviewed SHA-256 `BAC6B926B910B644D460128E8B4EC97DCF3120E527FA544AD1D0C1FF616AEBBA`.

[**E3**] *SFA 17-System Corpus Custody Manifest.* `work/paper-saturation-v1/evidence/what-software-systems-actually-receipt-govern-and-hide/sfa-17-corpus-manifest.v0.1.json`; SHA-256

3108754C519D0BA248C868E77818C386C5F78BA0A6700EA76DEF8CFAFA8E59BD.
It binds all seventeen reviewed

ANALYSIS.md files by path, size, and hash.

Ableman corpus

[**A1**] Adam Ableman. *Reasoning by Way of Receipts*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-04E427F6F4D94454;
source SHA-256
AF86A66CCD6A103DD75208500AE99148911A6453B6DFDF36A954279F21AB8489.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *The Epistemic Lineage Engine*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-E0EC3DC7EABF5060;
source SHA-256
CDD74D9AFAB6532645E68F7F6C9F0F7A9E472D9B8E717290522566FF8DC01F11.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *The Three Hash Rule*. Ableman Corpus current edition; work identity PAPER.PUB.20788617;
source SHA-256
F6FA73D083B70711A8C817F7779751D1506FEBA0E356DB9805BB1570663731B5.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *A Formal Theory of Work Custody*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-C8F2F9BF38C45E29;
source SHA-256
A80657E21BA277ADF1C229A2AAE221963B97A62EA2AF3EBC36434A916A8EEA44.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Governance Witnesses*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-9E8DA0EFE860DE5D;
source SHA-256
8BF8C3646040F6E1F5222A8AA92C1106AA1F841E8A2C65402F468C83D59D06E5.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Sovereign Software*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-24C6159B98023A9C;

source SHA-256
`6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Constitutional Software Analysis (CSoftA): A Governance-Based Classification Method for Software Systems.* Ableman Corpus current edition; work identity `PAPER.PUB.20472196`;
source SHA-256
`3670B28B7C1AF5467BE4002C47B5F964F8FAF6F257125C2F84CE06904E4FD0DE`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

[**E4**] Adam Ableman *Semantic Fragmentation and Constitutional Structure.* Current reviewed head SHA-256
`4048182A492E8ED731505426801D7A89074DE7311C0B3D975ECD2A2478D845F8`.

Reference-status note. These are internal corpus sources, not independent scholarly corroboration. The current paper contains product-specific claims that require exact version and primary-source revalidation for a future evidence-qualified revision. No external research was conducted during this pass.

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *Semantic Debt.* Zenodo, 2026. doi:10.5281/zenodo.21111666

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Adam Ableman *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Zenodo, 2026. doi:10.5281/zenodo.21095303

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848564